

Failure handling (three errors, three reactions) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/custom_errors_practice.py`, under its `# Failure handling` section — the same file also holds the [Basic exercise](#), under a `# Basic` section. Run it with `cd practice && python custom_errors_practice.py`.

Builds on: the `# Basic` section of that same file — same habit, three error classes instead of one. Nothing to copy or import: scroll up in the file you already have.

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real codebase would actually write it). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You need 3 different custom error types, one function that can raise any of the 3 depending on what happened, and a piece of code that catches each one separately — not all together in one block. Each error should get its own, different reaction.

Think of a real example: placing an order can fail because the item is out of stock, the payment was declined, or the address is wrong — three different problems that should be handled differently.

Here are the exact pieces you need:

- Make 3 classes, each one built from `Exception`: `OutOfStockError`, `PaymentDeclinedError`, `InvalidAddressError`.
- One function takes an input (like a word describing what went wrong) and raises the matching error.
- A `try` block can have several `except` parts, checked one after another.
- Each `except` part should actually do something different — not just print a different word.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

You need 3 different custom error classes, one function that can raise any of the 3 depending on its input, and a caller with 3 *separate* `except` blocks — not one shared one. The whole point is that each error type gets its own, different reaction, not just its own printed message.

Think of a real example: a function processing an order might fail because the item is out of stock, the payment was declined, or the address is invalid — three genuinely different problems that a caller should handle differently.

- Define 3 classes, each inheriting from `Exception`: e.g. `OutOfStockError`, `PaymentDeclinedError`, `InvalidAddressError`.
- One function takes an input (like a status code, or a string describing what went wrong) and raises the matching error.
- A `try/except` block can have multiple `except` clauses, checked in order: `except OutOfStockError: ... except PaymentDeclinedError: ... except InvalidAddressError: ...`
- Each `except` block should do something *different* — not just print a different string, but actually react differently (like retrying, versus asking the user to fix something, versus just logging and stopping).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Ask yourself: what happens when a 4th error type shows up next month, and then a 10th? A chain of `except` blocks, one per error, is easy to read for 3 errors — but it gets harder to maintain as the list grows, especially if several errors genuinely deserve the *same* reaction. The deeper design question isn't just "catch these 3 errors separately" — it's "how do I organize error *types* and error *reactions* so that adding error #11 doesn't mean rewriting a giant `except` chain, and so that errors which should share a reaction actually can, without losing the ability to handle one specifically when it matters?"

Two extra pieces make this scale:

- **A shared base class:** `class OrderError(Exception): pass`, with all 3 specific errors inheriting from it (`class OutOfStockError(OrderError): pass`, and so on). A caller can then catch the whole family with one `except OrderError:`, or one specific member of it, depending on what it needs.
- **A dispatch table:** a plain `dict` mapping each exception *class* to the function that should handle it — `HANDLERS = {OutOfStockError: handle_stock, ...}`. Instead of one `except` block per error type, a single `except Exception as e:` looks up `HANDLERS[type(e)]` and calls it. Adding error #11 becomes "add one dict entry and one function," not "add another `except` block to a growing chain."

Difference between Basic, Intermediate, and Advanced: Basic gives you the ingredients in plain words. Intermediate shows the exact syntax — 3 classes, 3 raises, 3 `except` blocks, and explains what each piece of syntax does. Advanced asks what happens once "3" becomes "many," and introduces two new *shapes* for organizing the same problem — a class hierarchy, and a lookup table — either of which is a genuinely different structural choice from "write N `except` blocks," not just a fancier version of it. That's the difference between code that works for this exercise and code that scales to a real, growing system.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make 3 error types: OutOfStockError, PaymentDeclinedError, InvalidAddressError
```

```
function process_order(problem_type):
    if problem_type is "stock": raise OutOfStockError
    if problem_type is "payment": raise PaymentDeclinedError
    if problem_type is "address": raise InvalidAddressError
```

```
for each of the 3 problem types:
    try process_order with that type
    except OutOfStockError: say "try again later, tell the warehouse"
    except PaymentDeclinedError: say "ask for a different payment method"
    except InvalidAddressError: say "ask the customer to fix their address"
```

```
# practice/custom_errors_practice.py – Failure handling section
class OutOfStockError(Exception):
    pass

class PaymentDeclinedError(Exception):
    pass

class InvalidAddressError(Exception):
    pass

def process_order(problem_type):
    if problem_type == "stock":
        raise OutOfStockError("item is out of stock")
    # add the other two raises yourself
```

Try finishing the function and the 3 `except` parts yourself first.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
define OutOfStockError, PaymentDeclinedError, InvalidAddressError, each as a kind

function process_order(problem_type):
    if problem_type is "stock":
        raise OutOfStockError
    if problem_type is "payment":
        raise PaymentDeclinedError
    if problem_type is "address":
        raise InvalidAddressError

for each of the 3 problem types:
    try to process_order with that type
    except OutOfStockError: print "try again later, notify the warehouse"
    except PaymentDeclinedError: print "ask the customer for a different payment"
    except InvalidAddressError: print "ask the customer to fix their address"
```

```
# practice/custom_errors_practice.py – Failure handling section
class OutOfStockError(Exception):
    pass

class PaymentDeclinedError(Exception):
    pass

class InvalidAddressError(Exception):
    pass

def process_order(problem_type: str) -> None:
    if problem_type == "stock":
        raise OutOfStockError("item is out of stock")
    # add the other two raises yourself
```

Try finishing the function and the 3-branch `except` block yourself before checking the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```

define OrderError as a kind of Exception (the shared base)
define OutOfStockError, PaymentDeclinedError, InvalidAddressError, each as a kind of OrderError

build a lookup table:
    HANDLERS = {
        OutOfStockError: a function that says "try again later, tell the warehouse"
        PaymentDeclinedError: a function that says "ask for a different payment method"
        InvalidAddressError: a function that says "ask the customer to fix their address"
    }

for each of the 3 problem types:
    try process_order with that type
    except Exception as e:
        look up HANDLERS by the exact type of e
        call whichever function was found, passing it e

```

```

# practice/custom_errors_practice.py – Failure handling section
class OrderError(Exception):
    pass

class OutOfStockError(OrderError):
    pass

class PaymentDeclinedError(OrderError):
    pass

class InvalidAddressError(OrderError):
    pass

def process_order(problem_type: str) -> None:
    if problem_type == "stock":
        raise OutOfStockError("item is out of stock")
    # add the other two raises yourself

HANDLERS = {
    OutOfStockError: None,          # replace with a real handler function
    PaymentDeclinedError: None,     # replace with a real handler function
    InvalidAddressError: None,      # replace with a real handler function
}

```

```
InvalidAddressError: None,    # replace with a real handler function  
}
```

Fill in 3 small handler functions, finish `HANDLERS`, and write the single `except Exception as e:` block that looks up `HANDLERS[type(e)]` and calls it. Then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same underlying idea (3 error types, 3 different reactions) at 3 completeness levels, shown here both as pseudocode and as near-complete code — Basic proves the concept works, Intermediate adds the type contract real Python expects. Basic and Intermediate's pseudocode both hard-code 3 separate `except` lines. Advanced restructures the dispatch itself: its pseudocode builds a lookup table once, so the `try/except` block itself never changes size again no matter how many error types get added — only the table grows — which is what makes the design still hold up once there are more than 3 error types.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)